Modern Database Systems

MASTER DIGITAL SCIENCE SOSE 2024

10.07.2024

Anh Thu Bui
Natalie Hußfeldt

Intro and Use Case

Universities should use data insights to enhance graduation rates and enriche the educational experience.

- Identify students in specific courses.
- Track student interactions with learning materials.
- Correlate interactions with grades.
- Analyze data to flag at-risk students and recommend personalized learning materials.
- Calculate average clicks and compare individual activities.
- Evaluate shared courses among students.

DB

PostgreSQL

- Offline, local

Neo4j AuraDB

- online, AuraDB free instance

Aim

Improve learning outcomes and prevent drop-outs.

Database Selection

PostgreSQL/ Relational Databases:

- PostgreSQL excels at structured data and ACID compliance for transactional applications.

Graph Databases (Neo4j):

- Ideal for university data with interconnected students, courses, grades, and materials.
- **Optimized Queries:** Neo4j simplifies complex queries like identifying common courses and tracking interactions.
- **Flexibility:** Neo4j adapts easily to new data types and relationships, crucial for dynamic educational environments.

Data

File name	Size
courses.csv	250 B
vle.csv	161 KB
assessments.csv	6 KB
studentInfo.csv	456 KB
studentAssessment.csv	2.2 MB
studentVle.csv	208.6 MB

Kaggle OULAD:

- Anonymized data on courses, students, and VLE interactions.

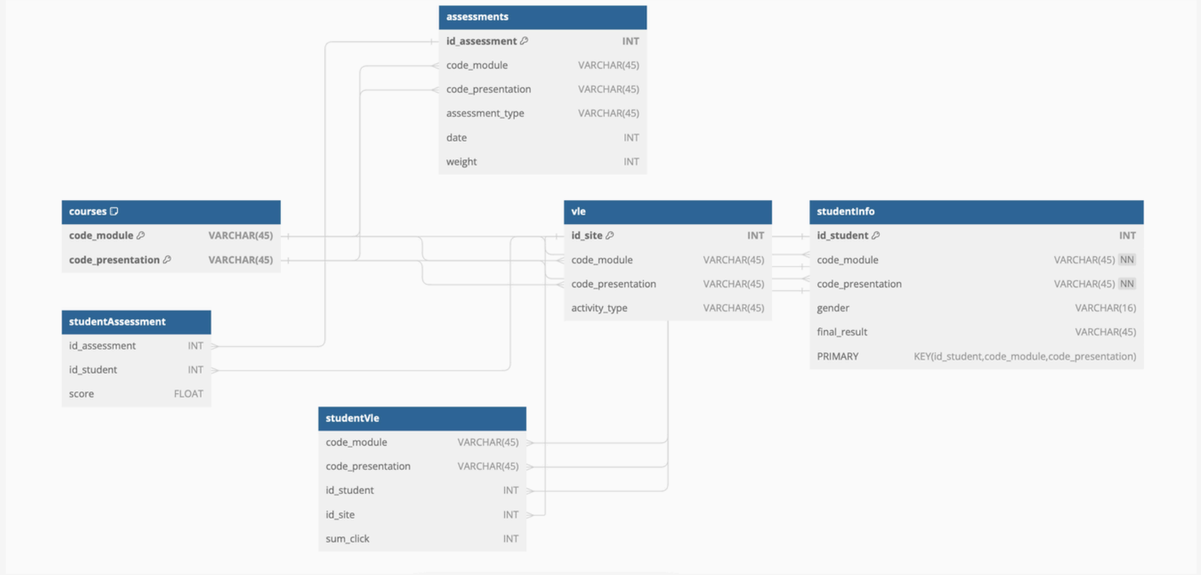
Data features

- Records of student enrolment, learning material usage, assessment performance.
- Facilitates research on online learning behavior

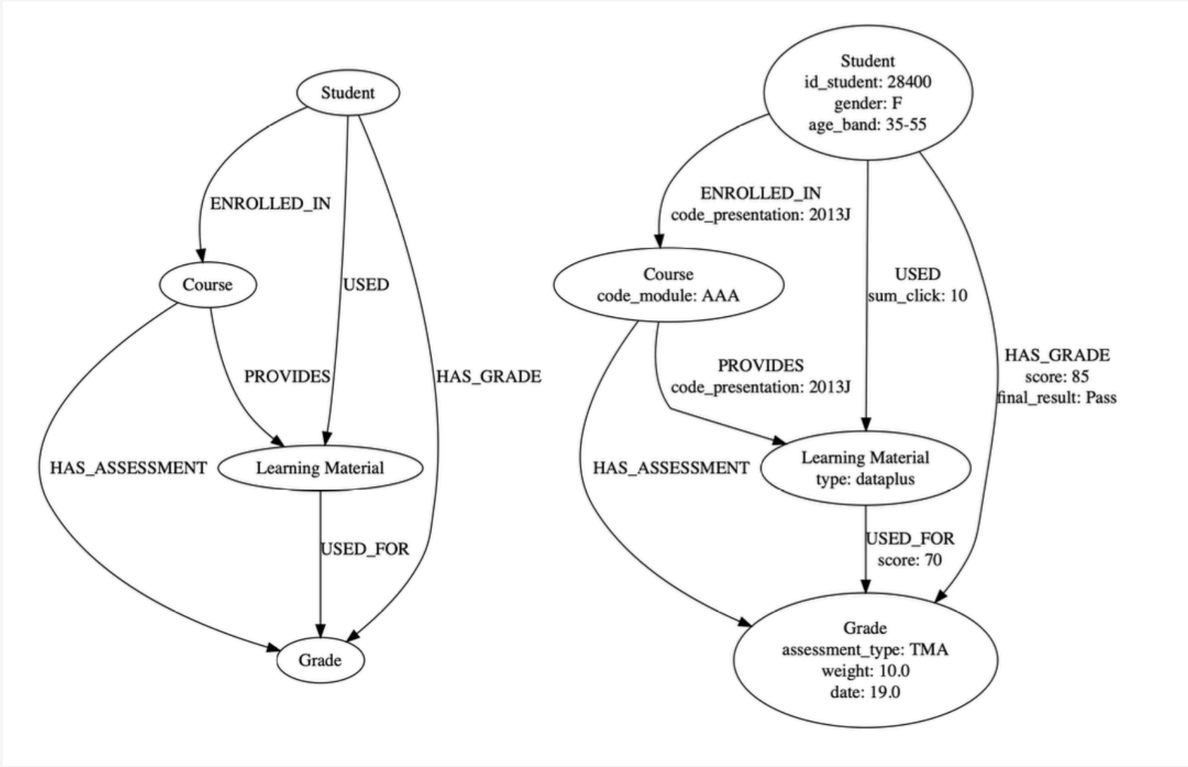
Structure

- Dataset stored in CSV files, connected through unique identifiers.
- Initial dataset size reduced for NoSQL database compatibility

Data Models



PostgreSQL



Neo4j

Queries

- Focus on these queries; may need additional queries for data deletion or updates.

Writing Query 1:
Inserts and updates student, course, material, and assessment records each term.

Reading Query 1:
Aggregates learning material clicks to analyze student engagement and performance correlations. Recommends materials clicked by former top students.

Reading Query 2:
The goal is to provide early alerts for students at risk by identifying those with lower than average engagement metrics.

Reading Query 3:
Connects students sharing common courses to recommend study partners.

Queries

Reading Query 4:
Lists students in a course for analyzing engagement and performance.

Reading Query 1: Recommendation of Learning Materials Based on Top Students

```sql
WITH QualifiedStudents AS (
    SELECT si.id_student
    FROM studentInfo si
    JOIN studentAssessment sa ON si.id_student = sa.
          id_student
    JOIN courses c ON si.code_module = c.code_module AND si.
          code_presentation = c.code_presentation
    WHERE c.code_module = 'AAA' AND LEFT(si.
          code_presentation, 4)::INT > 2013 AND sa.score >
          50.0
),
MostUsedMaterial AS (
    SELECT sv.id_student, v.activity_type, SUM(sv.sum_click)
           AS total_clicks
    FROM studentVle sv
    JOIN vle v ON sv.id_site = v.id_site
    WHERE sv.id_student IN (SELECT id_student FROM
          QualifiedStudents)
    GROUP BY sv.id_student, v.activity_type
    ORDER BY total_clicks DESC
)
SELECT activity_type, sum(total_clicks) AS clicks_total
FROM MostUsedMaterial
GROUP BY activity_type
ORDER BY clicks_total DESC;
```

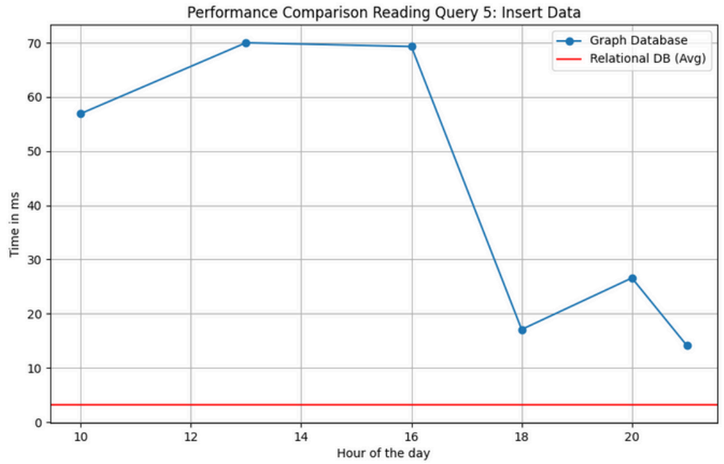
Listing 3: SQL Query

```cypher
// First MATCH clause to filter relevant courses and
      students
MATCH (c:Course {code_module: "AAA"})
WITH c

// Second MATCH clause to filter relevant ENROLLED_IN
      relationships
MATCH (s:Student)-[e:ENROLLED_IN]->(c)
WHERE toInteger(substring(e.code_presentation, 0, 4)) > 2013
WITH s, c

// Third MATCH clause to filter relevant HAS_GRADE
      relationships
MATCH (s)-[r:HAS_GRADE]->(g:Grade)-[:HAS_ASSESSMENT]-(c)
WHERE r.score > 50.0
WITH COLLECT(DISTINCT s) AS students

// Unwind to process the collected students further
UNWIND students AS s

// Fourth MATCH clause to process USED relationships and
      aggregate clicks
MATCH (s)-[u:USED]->(m:Learning_Material)
RETURN m.type as materialType, sum(u.sum_click) as
      total_clicks
ORDER BY total_clicks DESC;
```

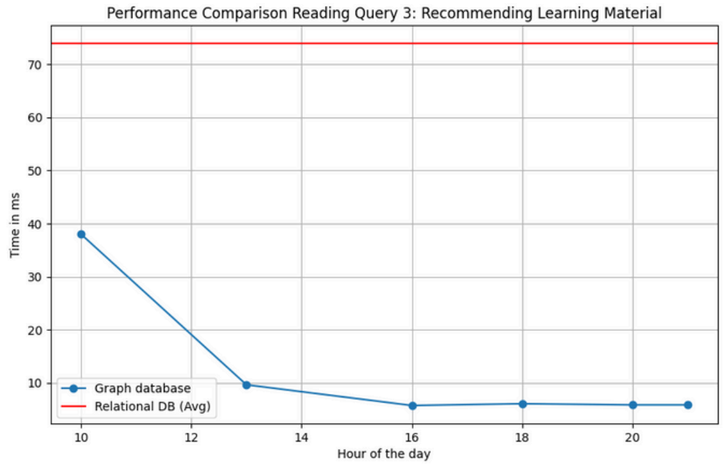
Listing 4: Cypher Query

Results

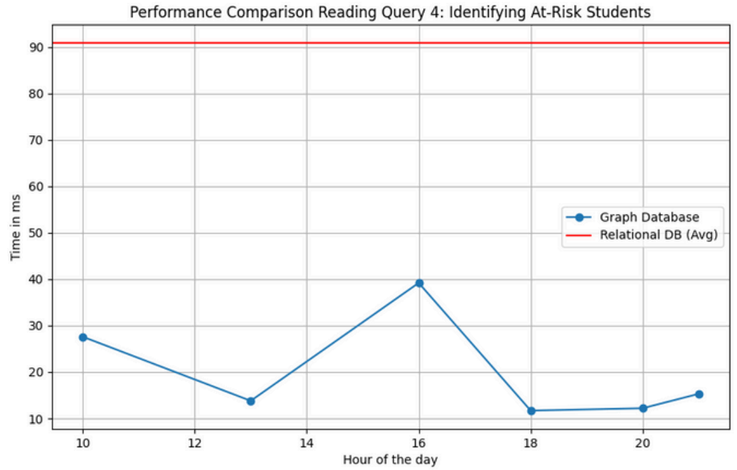
Writing Query



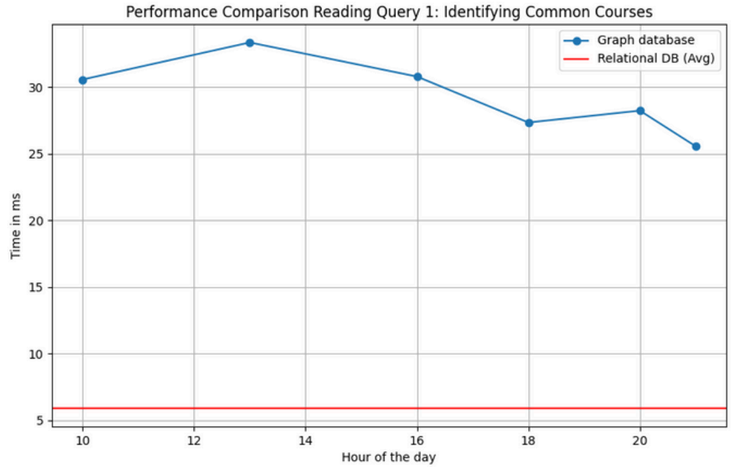
Recommendations



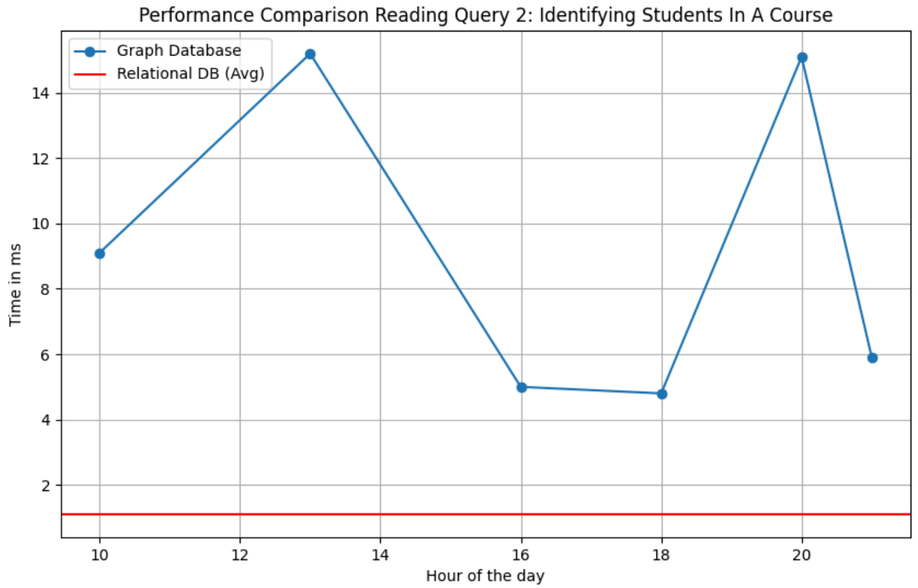
Risk Students



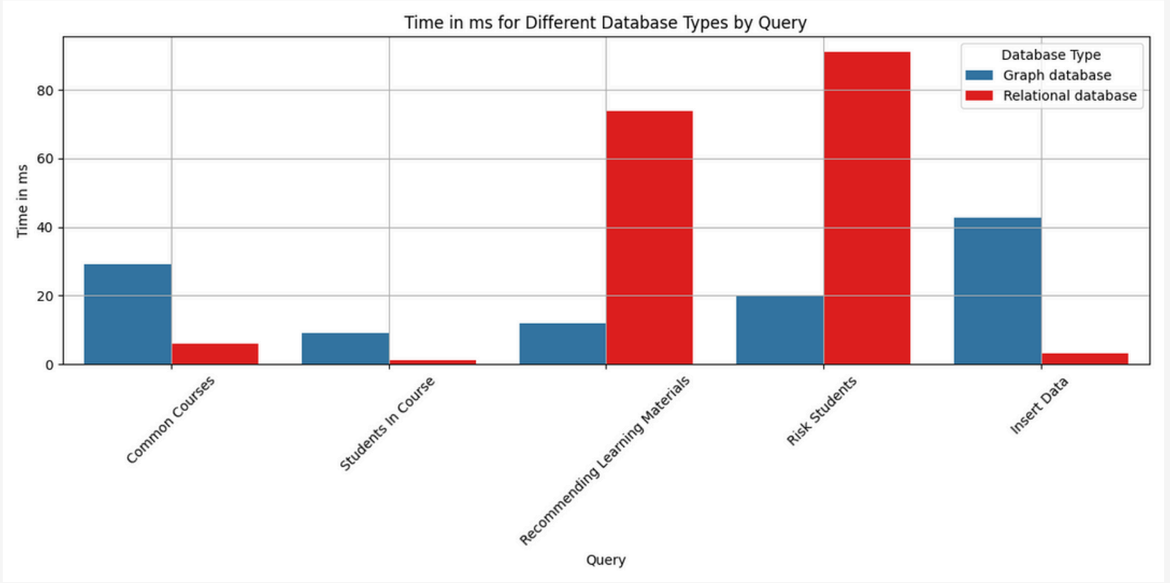
Common Courses



Results



Students In A Course



Performance Comparison

- **Query Efficiency:**
  - graph database excels in complex queries (recommendations and risk identification)
  - relational database outperforms in simple queries (data insertion, querying students)

Discussion

Limitations:

- Tests on a limited dataset.
- Neo4j AuraDB Free limited to 200k nodes/400k relationships.
- Online hosting may introduce latency.
- Neo4j's initial queries slower due to disk access before caching.

Insert Performance:

- Graph DB response times: 14.1 ms - 70 ms; prioritizes efficient data analysis.
- Relational DB insert times increase with table size.

Graph DB Advantages:

- .Efficient for suggesting study materials and identifying at-risk students.
- Handles complex relationships and deep queries efficiently.

Relational DB Advantages:

- Faster for simple queries with fewer relationships.
- Optimized for direct access and join operations.

Theoretical Outlook

01

Vendor Lock-In:

- Neo4j supports multiple programming languages
- Cypher, is widely used
- Migration Tools, Data Import Techniques

02

Cluster Arrangement:

- Neo4j uses a master-slave setup ensuring high availability and minimal downtime.

03

Scalability:

- Neo4j's Autonomous Clustering supports horizontal scaling for higher throughput.
- Sharding with Neo4j Fabric: Enables storage of large graphs in smaller independent graphs for easy scale-out. Maintains performance regardless of graph size, dependent on query complexity.

04

CAP Theorem:

- Neo4j prioritizes consistency and partition tolerance over availability.

05

Schema Changes:

- Adding new nodes and relationships usually doesn't affect existing queries.
- Simplifies changing existing relationships or node structures compared to other databases.

Conclusion

Neo4j Strengths

- ideal for managing complex relationships, real-time analytics and personalized learning in educational settings

Neo4j Weakness

- **Data Insertion**
  - less efficient than PostgreSQL for data insertion tasks (at the time)

Optimization Recommendations

- data inserts via batch processing during low activity periods
- use of tools like LOAD_csv for effective bulk data import
- consideration of upgrading to paid version